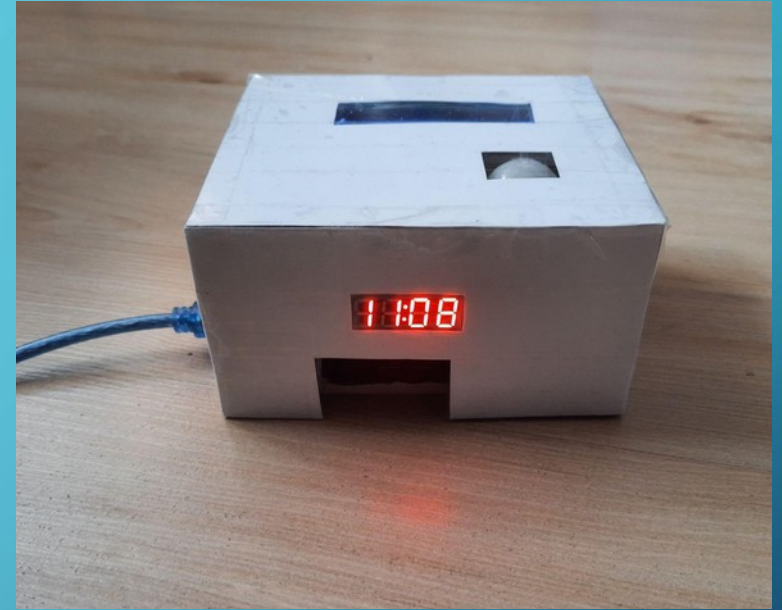


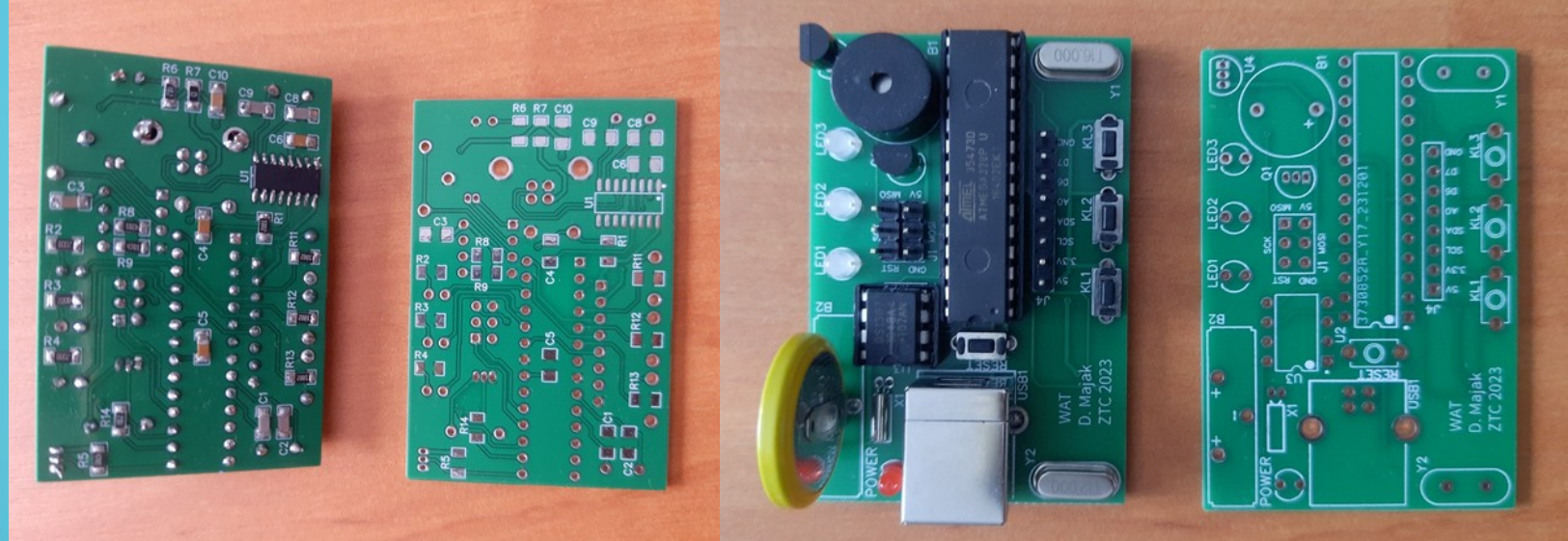
PREZENTACJA BUDZIKA



PRZEDMIOT: PROJEKTOWANIE SYSTEMÓW CYFROWYCH
AUTOR: DOMINIK MAJAK WEL21EC1S1

ZAŁOŻENIA

- projekt płytki budzika;
- polutowanie elementów;
- napisanie kodu zgodnie z założeniami;
- przeprowadzenie testów.



FUNKCJONALNOŚCI

- wskazywanie aktualnej godziny i daty;
- odczyt aktualnego wskazania temperatury (skala Celsjusza i Farenheita);
- ustawienie godziny alarmu i opcjonalnej drzemki;
- zmiana trybu dzwonienia budzika (ciągły, 0.5 s, 0.25 s, 0.1 s);
- **wyłączenie alarmu i drzemki możliwe po wykryciu ruchu (bez dotykania budzika);**
- **zmiana języka (polski, angielski).**

MECHANIZM ZMIANY STANÓW

```
void loop() {  
  
    uint8_t B1_val = digitalRead(BTN1);  
    uint8_t B2_val = digitalRead(BTN2);  
    uint8_t B3_val = digitalRead(BTN3);  
    uint8_t Pir = digitalRead(PIR);  
    display.setBrightness(0x0f);  
  
    switch (currentState) {
```

```
        if (currentState != nextState) {  
            currentState = nextState;  
            lcd.clear();  
            menuCounter = 1;  
            temperaturePrinted = false;  
            buzzerOn = 0;  
            buzzerOnNap = 0;  
        }  
    }
```

ODCZYT AKTUALNEJ GODZINY I DATY

```
case TIME_STATE:

    hour_set_alarm = 0;
    minute_set_alarm = 0;
    alarmSet = 0;
    napSet = 0;
    minute_set_nap = 0;
    menuCounter = 1;
    lastActiveAlarm = 0;
    digitalWrite(LED1, LOW);
    digitalWrite(LED2, LOW);
    digitalWrite(LED3, LOW);

    showDateLCD();
    showTime7Seg();
    digitalWrite(BUZZER, LOW);

    if (B1_val==0 && B2_val==1 && B3_val==0) {
        nextState = TEMP_STATE;
    }
    else if (B1_val==1 && B2_val==0 && B3_val==0) {
        nextState = SETTINGS_STATE;
        delay(intentionalDelay);
    }
    else {
        nextState = TIME_STATE;
    }

break;
```

```
void showTime7Seg () {
    DateTime now = rtc.now();

    // wyświetlanie czasu na wyświetlaczu 7-segmentowym
    uint8_t decimalPartHour = now.hour() / 10;
    uint8_t unityPartHour = now.hour() % 10;
    uint8_t decimalPartMinute = now.minute() / 10;
    uint8_t unityPartMinute = now.minute() % 10;

    sevenSegData[0] = display.encodeDigit(decimalPartHour);
    sevenSegData[1] = display.encodeDigit(unityPartHour);
    if (now.second() % 2 == 0) {
        sevenSegData[1] |= B10000000;    // Ustawia bit dla dwukropka
    }
    sevenSegData[2] = display.encodeDigit(decimalPartMinute);
    sevenSegData[3] = display.encodeDigit(unityPartMinute);

    display.setSegments(sevenSegData);
}
```

ODCZYT TEMPERATURY

```
case TEMP_STATE:

    showTemp();
    showTime7Seg();

    if (B1_val==0 && B2_val==0 && B3_val==1) {
        nextState = TIME_STATE;
    }
    else if (B1_val==1 && B2_val==0 && B3_val==0) {
        nextState = SETTINGS_STATE;
    }
    else {
        nextState = TEMP_STATE;
    }

    break;
```

```
void showTemp() {
    tempSensor.requestTemperatures();           // wyśle polecenie odczytu temperatury
    float newTempCelsius = tempSensor.getTempCByIndex(0); // odczytuje temperaturę w stopniach Celsjusza
    float newTempFahrenheit = newTempCelsius * 9.0 / 5.0 + 32.0;

    // sprawdza, czy temperatura się zmieniła
    if (newTempCelsius != tempCelsius || !temperaturePrinted) {
        tempCelsius = newTempCelsius; // aktualizuje poprzednią temperaturę

        lcd.clear();
        lcd.setCursor(0, 0);
        if (lang == pl) {
            lcd.print("Temperatura:");
        } else {
            lcd.print("Temperature:");
        }

        lcd.setCursor(0, 1); // ustawia kursor w pozycji 0 w drugiej linii
        lcd.print(tempCelsius);
        lcd.print(char(223));
        lcd.print("C");

        lcd.setCursor(9, 1); // ustawia kursor w pozycji 9 w drugiej linii
        lcd.print(newTempFahrenheit);
        lcd.print(char(223));
        lcd.print("F");

        temperaturePrinted = true; // ustawia zmienną, że temperatura została wydrukowana
    }
}
```

PORUSZANIE SIĘ PO MENU

```
case SETTINGS_STATE:

    displayMenu(B2_val);
    showTime7Seg();

    if (B1_val == 0 && B2_val == 0 && B3_val == 1) {
        nextState = TIME_STATE;
    } else if (B1_val == 1 && B2_val == 0 && B3_val == 0 &&
menuCounter == 1) {
        nextState = SET_ALARM_STATE;
        delay(intentionalDelay);
    } else if (B1_val == 1 && B2_val == 0 && B3_val == 0 &&
menuCounter == 2) {
        nextState = SET_BUZZER_MODE_STATE;
        delay(intentionalDelay);
    } else if (B1_val == 1 && B2_val == 0 && B3_val == 0 &&
menuCounter == 3) {
        nextState = SET_LANGUAGE_STATE;
        delay(intentionalDelay);
    } else {
        nextState = SETTINGS_STATE;
    }

    break;
```

```
void displayMenu(uint8_t Btn) {

    buttonRelease(Btn);

    if (menuCounter==1) {
        if (lang == pl) {
            lcd.setCursor(0, 0);
            lcd.print("Ustaw alarm");
            lcd.setCursor(0, 1);
            lcd.print("KL1 -> zatwierdz");
        } else {
            lcd.setCursor(0, 0);
            lcd.print("Set alarm");
            lcd.setCursor(0, 1);
            lcd.print("KL1 ->   select");
        }
    }
    else if (menuCounter==2) {
        if (lang == pl) {
            lcd.setCursor(0, 0);
            lcd.print("Zmien sygnał");
            lcd.setCursor(0, 1);
            lcd.print("KL1 -> zatwierdz");
        } else {
            lcd.setCursor(0, 0);
            lcd.print("Change signal");
            lcd.setCursor(0, 1);
            lcd.print("KL1 ->   select");
        }
    }
}
```

```
    else if (menuCounter==3) {
        if (lang == pl) {
            lcd.setCursor(0, 0);
            lcd.print("Zmien jezyk");
            lcd.setCursor(0, 1);
            lcd.print("KL1 -> zatwierdz");
        } else {
            lcd.setCursor(0, 0);
            lcd.print("Change language");
            lcd.setCursor(0, 1);
            lcd.print("KL1 ->   select");
        }
    }
    else {
        lcd.setCursor(0, 0);
        lcd.print("Nie ma");
        lcd.setCursor(0, 1);
        lcd.print("taliej opcji");
    }

    if (menuCounter > 3) {
        menuCounter = 1;
    }
}
```

ZMIANA JĘZYKA NA WYŚWIETLACZU LCD

```
case SET_LANGUAGE_STATE:

    changeLanguage(B2_val);
    showTime7Seg();

    if ( B1_val == 1 && B2_val == 0 && B3_val == 0 ) {
        nextState = SETTINGS_STATE;
        delay(intentionalDelay);
    }
    else {
        nextState = SET_LANGUAGE_STATE;
    }
    break;
```

```
void changeLanguage(uint8_t Btn) {
    buttonReleaseLang(Btn);

    if (menuCounterLang==1) {
        lcd.setCursor(0, 0);
        lcd.print("polski");
        lcd.setCursor(0, 1);
        lcd.print("KL1 -> zatwierdz");
        lang = pl;
    }
    else if (menuCounterLang==2) {
        lcd.setCursor(0, 0);
        lcd.print("English");
        lcd.setCursor(0, 1);
        lcd.print("KL1 -> select");
        lang = eng;
    }
    else {
        lcd.setCursor(0, 0);
        lcd.print("Nie ma");
        lcd.setCursor(0, 1);
        lcd.print("takiej opcji");
    }

    if (menuCounterLang > 2) {
        menuCounterLang = 1;
    }
}
```

ZMIANA TRYBU DZWONIENIA BUDZIK

```
case SET_BUZZER_MODE_STATE:

    changeBuzzerMode(B2_val);
    showTime7Seg();

    if ( B1_val == 1 && B2_val == 0 && B3_val == 0 ) {
        nextState = SETTINGS_STATE;
        delay(intentionalDelay);
    }
    else {
        nextState = SET_BUZZER_MODE_STATE;
    }
    break;
```

```
void changeBuzzerMode(uint8_t Btn) {
    buttonReleaseBuzzer(Btn);

    if (menuCounterBuzzer==1) {
        if (lang == pl) {
            lcd.setCursor(0, 0);
            lcd.print("Ciagly");
            lcd.setCursor(0, 1);
            lcd.print("KL1 -> zatwierdz");
        } else {
            lcd.setCursor(0, 0);
            lcd.print("Constant");
            lcd.setCursor(0, 1);
            lcd.print("KL1 -> select");
        }
        mode = 1;
    }
    else if (menuCounterBuzzer==2) {
        if (lang == pl) {
            lcd.setCursor(0, 0);
            lcd.print("Co 0.5 s");
            lcd.setCursor(0, 1);
            lcd.print("KL1 -> zatwierdz");
        } else {
            lcd.setCursor(0, 0);
            lcd.print("Every 0.5 sec");
            lcd.setCursor(0, 1);
            lcd.print("KL1 -> select");
        }
        mode = 2;
    }
}
```

```
    else if (menuCounterBuzzer==3) {
        if (lang == pl) {
            lcd.setCursor(0, 0);
            lcd.print("Co 0.25 s");
            lcd.setCursor(0, 1);
            lcd.print("KL1 -> zatwierdz");
        } else {
            lcd.setCursor(0, 0);
            lcd.print("Every 0.25 sec");
            lcd.setCursor(0, 1);
            lcd.print("KL1 -> select");
        }
        mode = 3;
    }
    else if (menuCounterBuzzer==4) {
        if (lang == pl) {
            lcd.setCursor(0, 0);
            lcd.print("Co 0.1 s");
            lcd.setCursor(0, 1);
            lcd.print("KL1 -> zatwierdz");
        } else {
            lcd.setCursor(0, 0);
            lcd.print("Every 0.1 sec");
            lcd.setCursor(0, 1);
            lcd.print("KL1 -> select");
        }
        mode = 4;
    }
    else {
        lcd.setCursor(0, 0);
        lcd.print(F("Nie ma"));
        lcd.setCursor(0, 1);
        lcd.print("takiej opcji");
    }

    if (menuCounterBuzzer < 1) {
        menuCounterBuzzer = 4;
    }
}
```

USTAWIENIE GODZINY ALARMU I DRZEMKI

```
case SET_ALARM_STATE:

    setAlarm(B1_val, B2_val);

    if (B1_val==0 && B2_val==0 && B3_val==1) {
        nextState = SET_NAP_STATE;
        delay(intentionalDelay);
    }
    else {
        nextState = SET_ALARM_STATE;
    }

    break;
```

```
void setAlarm(uint8_t B1_val, uint8_t B2_val) {
    if (lang == pl) {
        lcd.setCursor(0, 0);
        lcd.print("KL1 -> godzina+1");
        lcd.setCursor(0, 1);
        lcd.print("KL2 -> minuta+1");
    } else {
        lcd.setCursor(0, 0);
        lcd.print("KL1 -> hour+1");
        lcd.setCursor(0, 1);
        lcd.print("KL2 -> minute+1");
    }

    DateTime now = rtc.now();
    alarmSet = 1;

    if (B1_val == 1 && B2_val == 0) {
        if (hour_set_alarm < 23) {
            hour_set_alarm++;
            delay(changeHourMinute);
        } else {
            hour_set_alarm = 0;
            delay(changeHourMinute);
        }
    }
}
```

```
if (B1_val == 0 && B2_val == 1) {
    if (minute_set_alarm < 59) {
        minute_set_alarm++;
        delay(changeHourMinute);
    } else {
        minute_set_alarm = 0;
        delay(changeHourMinute);
    }
}

sevenSegData[0] = display.encodeDigit(hour_set_alarm / 10);
sevenSegData[1] = display.encodeDigit(hour_set_alarm % 10);
sevenSegData[1] |= B10000000; // Ustawia bit dla dwukropka
sevenSegData[2] = display.encodeDigit(minute_set_alarm / 10);
sevenSegData[3] = display.encodeDigit(minute_set_alarm % 10);

display.setSegments(sevenSegData);
}
```

CZEKANIE NA ALARM

```
case ALARM_STATE:
    checkForAlarmLCD();
    checkForAlarm();
    turnOnBuzzer();

    if (B1_val==0 && B2_val==1 && B3_val==0 && buzzerOn==0) {
        nextState = TEMP_WITH_ALARM_STATE;
        delay(intentionalDelay);
    }
    else if (B1_val==1 && B2_val==0 && B3_val==0 && buzzerOn==0) {
        nextState = ASK_FOR_TURN_OFF_STATE;
        delay(intentionalDelay);
    }
    else if ((B1_val==1 || B2_val==1 || B3_val==1 || Pir==1) &&
        buzzerOn==1) {
        if (minute_set_nap == 0) {
            nextState = TIME_STATE;
        } else {
            nextState = NAP_STATE;
            lastActiveAlarm = 1;
            delay(intentionalDelay);
        }
    }
    else {
        nextState = ALARM_STATE;
    }
    break;
```

```
void checkForAlarm () {
    DateTime now = rtc.now();
    uint8_t decimalPartHour = now.hour() / 10;
    uint8_t unityPartHour = now.hour() % 10;
    uint8_t decimalPartMinute = now.minute() / 10;
    uint8_t unityPartMinute = now.minute() % 10;

    sevenSegData[0] = display.encodeDigit(decimalPartHour);
    sevenSegData[1] = display.encodeDigit(unityPartHour);
    if (now.second() % 2 == 0) {
        sevenSegData[1] |= B10000000;
    }
    sevenSegData[2] = display.encodeDigit(decimalPartMinute);
    sevenSegData[3] = display.encodeDigit(unityPartMinute);

    display.setSegments(sevenSegData);

    if ( decimalPartHour == hour_set_alarm / 10 &&
        unityPartHour == hour_set_alarm % 10 &&
        decimalPartMinute == minute_set_alarm / 10 &&
        unityPartMinute == minute_set_alarm % 10 &&
        alarmSet==1
    )
    {
        buzzerOn = 1;
    }
}
```

WYŁĄCZENIE ALARMU PRZED CZASEM

```
case ASK_FOR_TURN_OFF_STATE:

    digitalWrite(LED1, HIGH);
    askDisplay();
    if(lastActiveAlarm == 0) {
        checkForAlarm();
        turnOnBuzzer();
    }
    else {
        checkForNap();
        buzzerNap();
    }

    if (B1_val==0 && B2_val==0 && B3_val==1 && buzzerOn==0) {
        if (lastActiveAlarm == 0) {
            nextState = ALARM_STATE;
            delay(intentionalDelay);
        }
        else if ( lastActiveAlarm == 1 ) {
            nextState = NAP_STATE;
            delay(intentionalDelay);
        }
    }
    else if (B1_val==0 && B2_val==1 && B3_val==0 && buzzerOn==0) {
        nextState = TIME_STATE;
        delay(intentionalDelay);
    }
    else if (B1_val==1 && B2_val==0 && B3_val==0 && buzzerOn==0 && buzzerOnNap==0) {
        nextState = SHOW_ALARM_HOUR_STATE;
        delay(intentionalDelay);
    }
}
```

```
else if ((B1_val==1 || B2_val==1 || B3_val==1 || Pir==1) && buzzerOn==1) {
    if (minute_set_nap == 0) {
        nextState = TIME_STATE;
        delay(intentionalDelay);
    } else {
        lastActiveAlarm = 1;
        nextState = NAP_STATE;
        delay(intentionalDelay);
    }
}
else if ((B1_val==1 || B2_val==1 || B3_val==1 || Pir==1) && buzzerOnNap==1) {
    nextState = TIME_STATE;
    delay(intentionalDelay);
}
else {
    nextState = ASK_FOR_TURN_OFF_STATE;
}

break;
```

Sprawdzenie godziny alarmu i drzemki

```
case SHOW_ALARM_HOUR_STATE:

    showAlarmHour();
    if(lastActiveAlarm == 0) {
        checkForAlarm();
        turnOnBuzzer();
    }
    else {
        checkForNap();
        buzzerNap();
    }

    if (B1_val==0 && B2_val==0 && B3_val==1 && buzzerOnNap==0 && buzzerOn==0) {
        if (lastActiveAlarm == 0) {
            nextState = ALARM_STATE;
            delay(intentionalDelay);
        }
        else if (lastActiveAlarm == 1) {
            lastActiveAlarm = 1;
            nextState = NAP_STATE;
            delay(intentionalDelay);
        }
    }
    else if ( (B1_val==1 || B2_val==1 || B3_val==1 || Pir==1) && buzzerOn==1 && minute_set_nap!=0) {
        nextState = NAP_STATE;
        delay(intentionalDelay);
    }
    else if ( (B1_val==1 || B2_val==1 || B3_val==1 || Pir==1) && buzzerOn==1 && minute_set_nap==0) {
        nextState = TIME_STATE;
        delay(intentionalDelay);
    }
    else if ( (B1_val==1 || B2_val==1 || B3_val==1 || Pir==1) && buzzerOnNap==1) {
        nextState = TIME_STATE;
        delay(intentionalDelay);
    }
    else {
        nextState = SHOW_ALARM_HOUR_STATE;
    }

    break;
```

```
void showAlarmHour() {
    lcd.setCursor(0, 0);
    lcd.print("Alarm:");
    if (hour_set_alarm > 9) {
        lcd.setCursor(11, 0);
        lcd.print(hour_set_alarm);
    } else {
        lcd.setCursor(12, 0);
        lcd.print(hour_set_alarm);
    }
    lcd.setCursor(13, 0);
    lcd.print(":");
    if (minute_set_alarm > 9) {
        lcd.setCursor(14, 0);
        lcd.print(minute_set_alarm);
    } else {
        lcd.setCursor(14, 0);
        lcd.print("0");
    }
    lcd.setCursor(15, 0);
    lcd.print(minute_set_alarm);
}
```

```
    lcd.setCursor(0, 1);
    if (lang == pl) {
        lcd.print("Drzemka:");
    } else {
        lcd.print("Nap:");
    }
    if (minute_set_nap < 10) {
        lcd.setCursor(11, 1);
        lcd.print(minute_set_nap);
        lcd.setCursor(13, 1);
        lcd.print("min");
    } else {
        lcd.setCursor(10, 1);
        lcd.print(minute_set_nap);
        lcd.setCursor(13, 1);
        lcd.print("min");
    }
}
```

MOJE PRZEMYŚLENIA I DOŚWIADCZENIA

- Elektronika się grzeje, dlatego czujnik temperatury należy umieszczać daleko od jej największego skupienia np. zaraz za obudową.
- Tanie układy czasu rzeczywistego ulegają łatwo rozkalibrowaniu.
- Poznałem nowe środowisko do projektowania płytek PCB i schematów ideowych.
- Poprawiłem umiejętności lutowania elementów THT i SMD.
- Miałem możliwość napisania programu systemu cyfrowego.

A decorative graphic on the left side of the slide, consisting of a network of light blue lines and small circles, resembling a circuit board or a stylized tree structure.

DZIĘKUJĘ ZA UWAGĘ